Sequential Handwritten Kannada Digit Classification

A Row-by-Row Sequential Modelling Study using Vanilla RNN, LSTM, and GRU

Ankit Saini (2522MA01)
Tiyas Chakrabortty (2522MA02)
Tanu Goliyan (2621MA05)
Murari Ray (2621MA07)

Department of Mathematics
Indian Institute of Technology Patna

09 October 2026

Contents

1 Introduction **3**

2 Objectives **3**

3 Dataset and Preprocessing **3**

3.1 Sequential representation 3

3.2 Data partition 4

3.3 Sample digit visualizations 4

4 Methodology **5**

4.1 Model architecture 5

4.2 Training and model selection 5

4.3 Classification error 6

5 Experimental Configuration **6**

6 Training and Validation Error **6**

7 Experimental Results **7**

7.1 Accuracy across configurations 7

7.2 Architecture-level comparison 8

7.3 Detailed evaluation of the selected configuration 8

7.4 Class-wise performance 9

8 Qualitative Visualizations and Comparative Analysis **10**

8.1 Sequential feature heatmaps 10

8.2 Comparative observations 10

8.3 Gradient-flow considerations 10

9 Discussion **11**

10 Conclusion **11**

11 Key Experimental Values **12**

1 Introduction

This report studies the sequential classification of handwritten Kannada numerals from ten classes, corresponding to digits 0–9. Each 28×28 grayscale image is treated as a sequence of 28 row vectors, so the recurrent model receives 28 time steps and 28 features per time step. This formulation allows recurrent architectures to process an image row by row rather than as a flattened vector.

Three recurrent architectures have been compared: a Vanilla Recurrent Neural Network (RNN), a Long Short-Term Memory network (LSTM), and a Gated Recurrent Unit network (GRU). Six configurations vary the recurrent hidden-unit count, dropout, and learning rate. The report summarizes error curves, accuracy across data splits, the selected model, its confusion matrix, sequential feature visualizations, and qualitative comparisons between the architectures.

2 Objectives

The objectives of the experiment are as follows:

(i) Preprocess handwritten Kannada digit images and represent each image as a row-wise sequence;

(ii) Train and compare Vanilla RNN, LSTM, and GRU classifiers under six hyperparameter configurations;

(iii) Plot training and validation classification error against epochs for each configuration;

(iv) Compare training, validation, and held-out test accuracy, selecting a configuration using validation accuracy;

(v) Report detailed evaluation metrics and the complete confusion matrix for the selected model; and

(vi) Visualize digit samples and sequential feature heatmaps and discuss convergence and gradient-flow mechanisms qualitatively.

3 Dataset and Preprocessing

The experiment uses a labelled Kannada-MNIST CSV dataset from: https://www.kaggle.com/c/Kannada-MNIST/data. The labelled input file used for these reported experiments contains 10,240 observations. Each observation has 784 grayscale pixel values and a label in $\{0, 1, \ldots, 9\}$. Pixel values are reshaped into a 28×28 image and normalized from $[0, 255]$ to $[0, 1]$ by dividing by 255.0.

3.1 Sequential representation

Let $I \in \mathbb{R}^{28\times28}$ denote a normalized image. The image is represented as a sequence

$$X = [\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_{28}]^T \in \mathbb{R}^{28\times28}, \tag{1}$$

where $\mathbf{x}_t \in \mathbb{R}^{28}$ is row t of the image. Thus every sample has 28 time steps, and each time step contains 28 pixel features. This row-wise sequence is passed to the recurrent layers.

3.2 Data partition

A stratified split was used to preserve approximate class proportions: 70% of the data for training, 15% for validation, and 15% for held-out testing. The exact counts are shown in Table 1.

Table 1: Dataset partition used in the experiment.

Subset	Number of samples	Share
Training	7,168	70%
Validation	1,536	15%
Held-out test	1,536	15%
Total	10,240	100%

3.3 Sample digit visualizations

Figures 1 and 2 show five sample handwritten images for each of the ten digit classes. Examples within the same class can vary in stroke thickness, slant, curvature, and pixel intensity. Such variations motivate learning a representation from the image sequence rather than using a manually specified pixel rule.

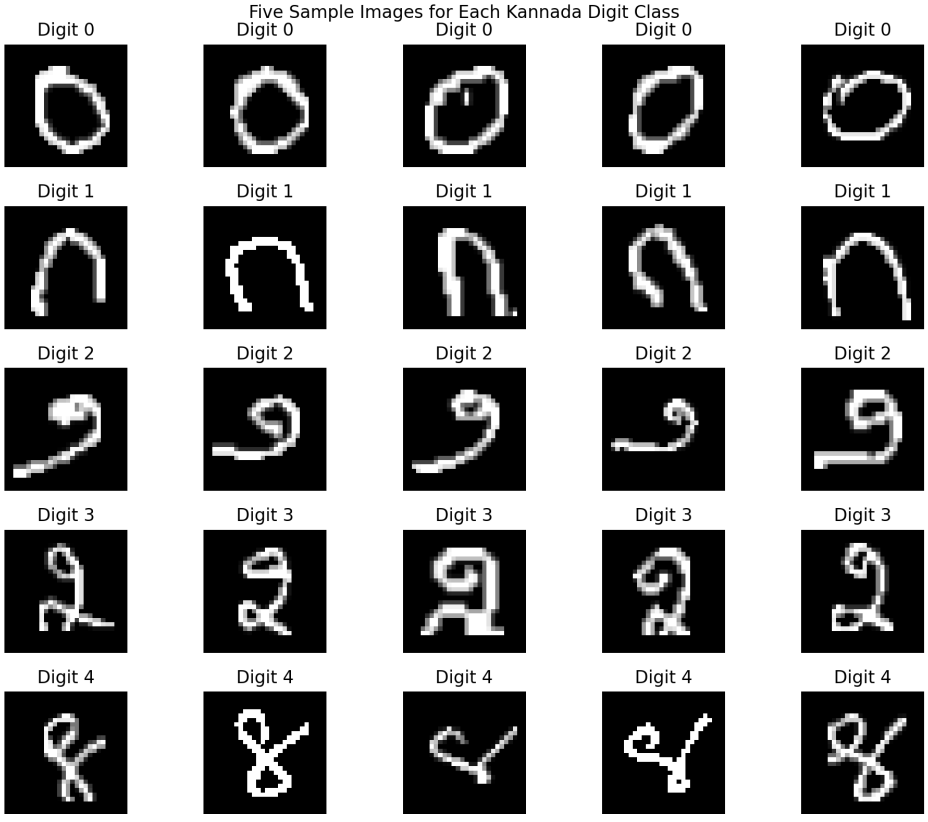


Figure 1: Five sample images from each Kannada digit class 0–4.

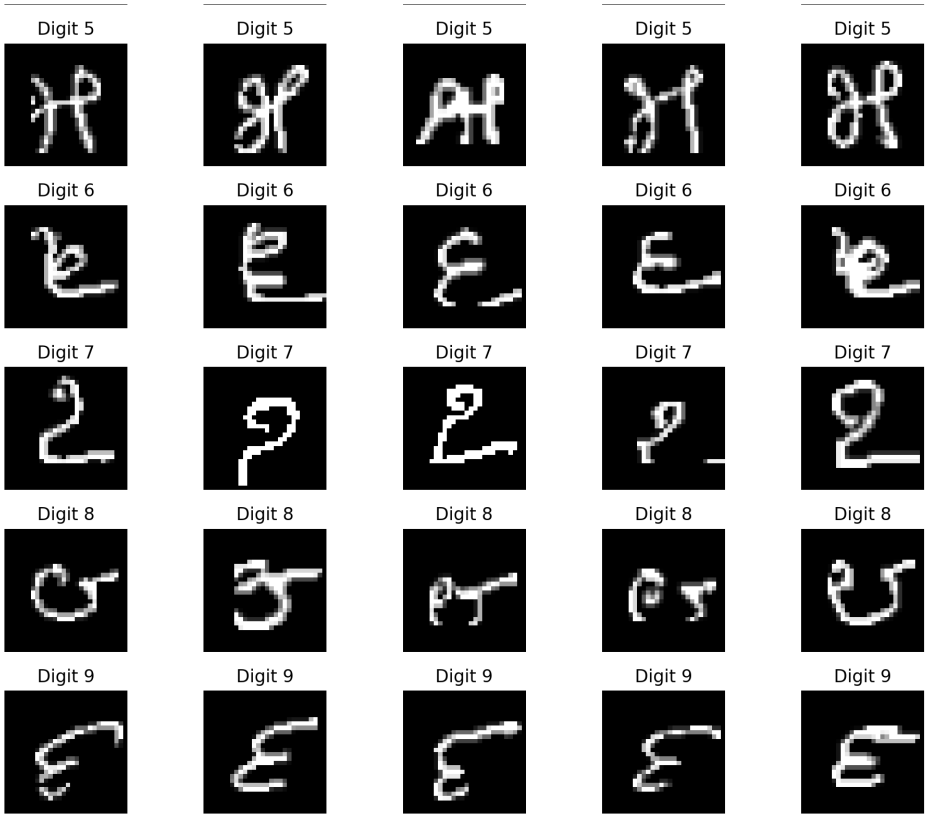


Figure 2: Five sample images from each Kannada digit class 5–9.

4 Methodology

4.1 Model architecture

Each network receives an input sequence of length 28, with 28 pixel features at every time step. A recurrent layer with either 64 or 128 hidden units processes the sequence. Its final representation is passed to a fully connected layer with 64 ReLU units and then to a 10-unit softmax output layer. The predicted digit is the class with the largest softmax probability.

The general architecture is

$$\begin{aligned}(28 \text{ time steps}, 28 \text{ features}) &\longrightarrow \text{RNN/LSTM/GRU} \\ &\longrightarrow \text{Dense}(64, \text{ReLU}) \\ &\longrightarrow \text{Dense}(10, \text{softmax}).\end{aligned}$$

4.2 Training and model selection

All models use the Adam optimizer, sparse categorical cross-entropy loss, a batch size of 128, and a maximum of 20 epochs. The random seed is 42. Early stopping monitors validation loss with patience 4 and restores the weights from the epoch with the best validation loss. The configuration with the highest validation accuracy is selected for detailed evaluation. Test accuracy is then reported as held-out performance rather than used to choose the configuration.

4.3 Classification error

For epoch e, the plotted classification error is defined as

$$E_e = 1 - \text{Accuracy}_e. \tag{2}$$

The saved curves therefore show classification error, not cross-entropy loss. Loss values are reported separately in the experiment table.

5 Experimental Configuration

Table 2 summarizes the common training settings, and Table 3 lists the six configurations compared in the experiment.

Table 2: Common experimental settings.

Parameter	Setting
Task	Ten-class sequential handwritten Kannada digit classification
Input representation	28 time steps × 28 pixel features
Normalization	Pixel intensity divided by 255.0
Recurrent layer	SimpleRNN, LSTM, or GRU; 64 or 128 units
Dropout	0.0 or 0.2 in the recurrent layer
Dense hidden layer	64 units with ReLU activation
Output layer	10 units with softmax activation
Loss	Sparse categorical cross-entropy
Optimizer	Adam
Batch size	128
Maximum epochs	20
Early stopping	Monitor validation loss; patience 4; restore best weights
Random seed	42
Data split	70% training, 15% validation, 15% test

Table 3: Hyperparameter configurations explored.

Configuration	Architecture	Units	Dropout	Learning rate	Epochs
`RNN_64_lr001`	RNN	64	0.0	0.001	18
`RNN_128_lr001`	RNN	128	0.2	0.001	17
`LSTM_64_lr001`	LSTM	64	0.0	0.001	19
`LSTM_128_lr0005`	LSTM	128	0.2	0.0005	20
`GRU_64_lr001`	GRU	64	0.0	0.001	20
`GRU_128_lr0005`	GRU	128	0.2	0.0005	20

6 Training and Validation Error

Figure 3 displays the training and validation classification error curves for all six configurations. The curves generally decrease rapidly during the early epochs and then flatten, indicating that most of the improvement occurs early in training. The Vanilla RNN configurations end with

higher errors than the best LSTM and GRU configurations. The 64-unit GRU has the lowest final training error among the tested configurations, while its validation error remains higher than its training error.

Small fluctuations or plateaus in validation curves near the end of training are visible. The difference between training and validation curves indicates a generalization gap, although the curves alone do not establish severe overfitting.

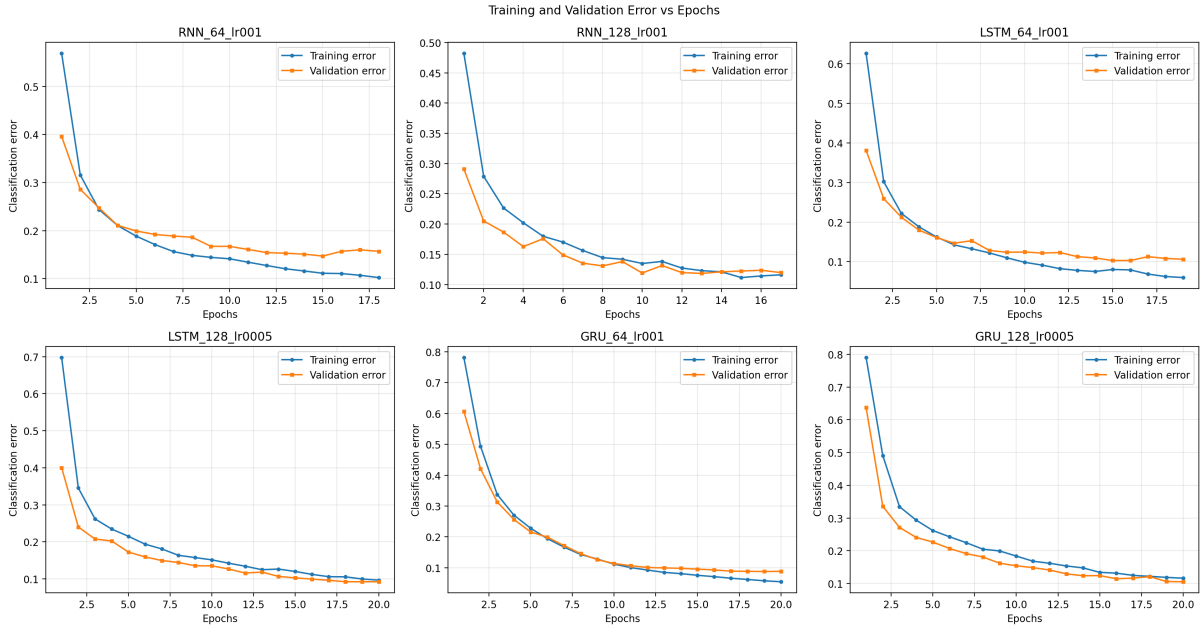


Figure 3: Training and validation classification error versus epochs for all six hyperparameter configurations.

Individual error-curve plots for every configuration are also supplied in the `figures/` directory of the source package.

7 Experimental Results

7.1 Accuracy across configurations

Table 4 reports final training, validation, and test accuracy for each trained configuration. The 64-unit GRU achieves the highest validation accuracy, 91.15%, and is selected for detailed evaluation. The 64-unit LSTM achieves the highest test accuracy among the six configurations, at 90.30%. The selected GRU has a lower test accuracy of 89.58%, but it is selected using validation accuracy rather than the held-out test results.

Table 4: Training, validation, and test accuracy for all configurations.

Configuration	Model	Training	Validation	Test
`RNN_64_lr001`	RNN	88.85%	84.90%	85.48%
`RNN_128_lr001`	RNN	90.85%	88.15%	89.00%
`LSTM_64_lr001`	LSTM	93.44%	89.71%	90.30%
`LSTM_128_lr0005`	LSTM	93.09%	90.76%	89.91%
`GRU_64_lr001`	GRU	94.36%	91.15%	89.58%
`GRU_128_lr0005`	GRU	91.00%	89.45%	88.61%

7.2 Architecture-level comparison

Figure 4 compares the best validation accuracy and best test accuracy obtained within each architecture. In this experiment, the GRU has the best validation score, while the LSTM has the best test score. The Vanilla RNN improves substantially with 128 units and dropout compared with its 64-unit configuration.

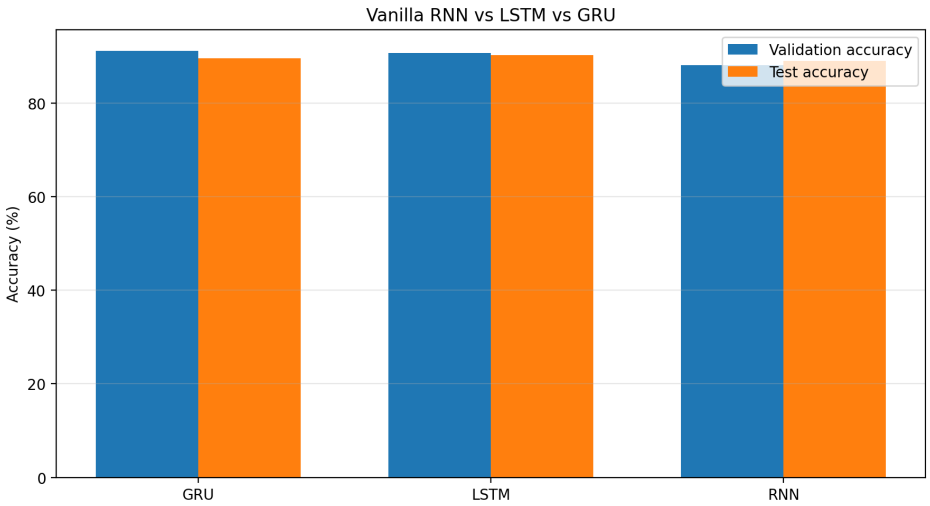


Figure 4: Best validation and test accuracies within each recurrent architecture.

7.3 Detailed evaluation of the selected configuration

The selected model is `GRU_64_lr001`, with 64 recurrent units, dropout 0.0, and learning rate 0.001. Its training accuracy is 94.36%, validation accuracy is 91.15%, and test accuracy is 89.58%. The confusion matrix contains 1,376 correct predictions out of 1,536 test samples, giving

$$\frac{1376}{1536} \times 100\% = 89.58\%.$$

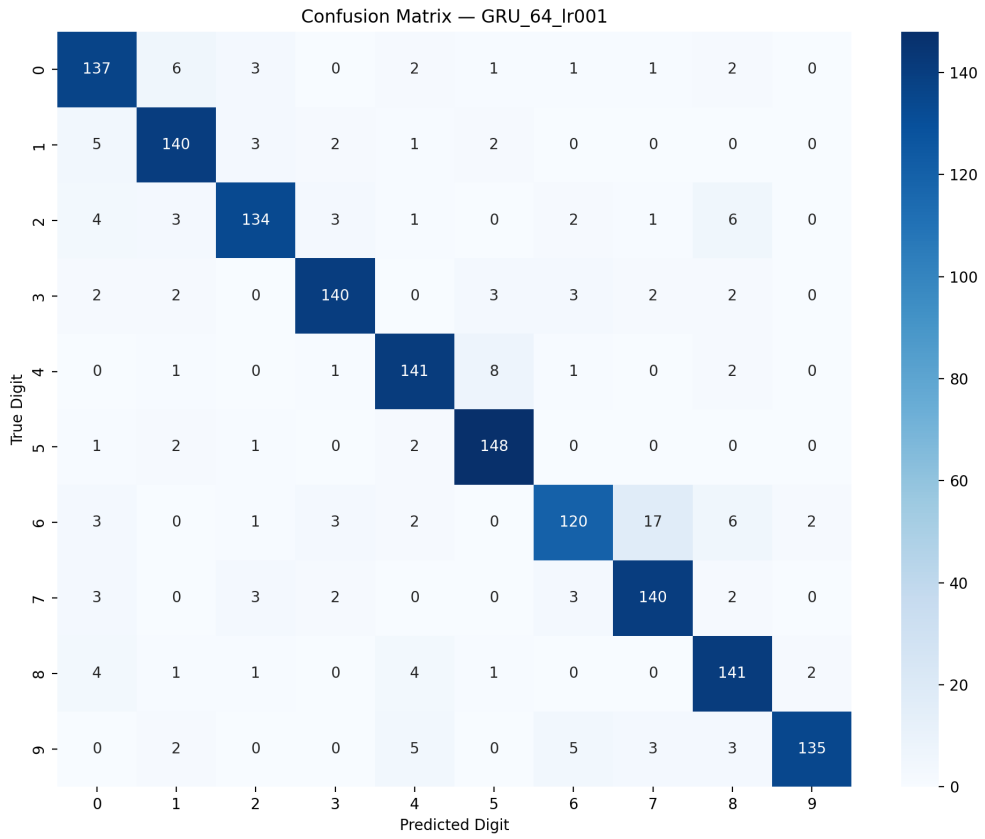


Figure 5: Complete test confusion matrix for the selected GRU configuration. Rows correspond to true classes and columns to predicted classes.

7.4 Class-wise performance

Table 5 shows precision, recall, F1-score, and support for each class based on the selected model's test confusion matrix. Class 5 has the highest recall (96.10%), while class 6 has the lowest recall (77.92%). Class 6 is frequently confused with class 7. The macro-average F1-score is approximately 89.56%.

Table 5: Class-wise test metrics for the selected GRU model.

Class	Precision	Recall	F1-score	Support
0	0.8616	0.8954	0.8782	153
1	0.8917	0.9150	0.9032	153
2	0.9178	0.8701	0.8933	154
3	0.9272	0.9091	0.9180	154
4	0.8924	0.9156	0.9038	154
5	0.9080	0.9610	0.9338	154
6	0.8889	0.7792	0.8304	154
7	0.8537	0.9150	0.8833	153
8	0.8598	0.9156	0.8868	154
9	0.9712	0.8824	0.9247	153
Macro average	0.8972	0.8958	0.8956	1,536
Weighted average	0.8972	0.8958	0.8956	1,536

8 Qualitative Visualizations and Comparative Analysis

8.1 Sequential feature heatmaps

Figure 6 presents one test example from each digit class as a 28×28 heatmap. The vertical axis represents the timestep (image row), and the horizontal axis represents the pixel feature within that row. Brighter cells correspond to higher normalized pixel intensities. This visualization shows the structure passed to the recurrent model at successive time steps. These are image-pixel heatmaps, not MFCC features; MFCC visualizations belong to the separate spoken-digit task.

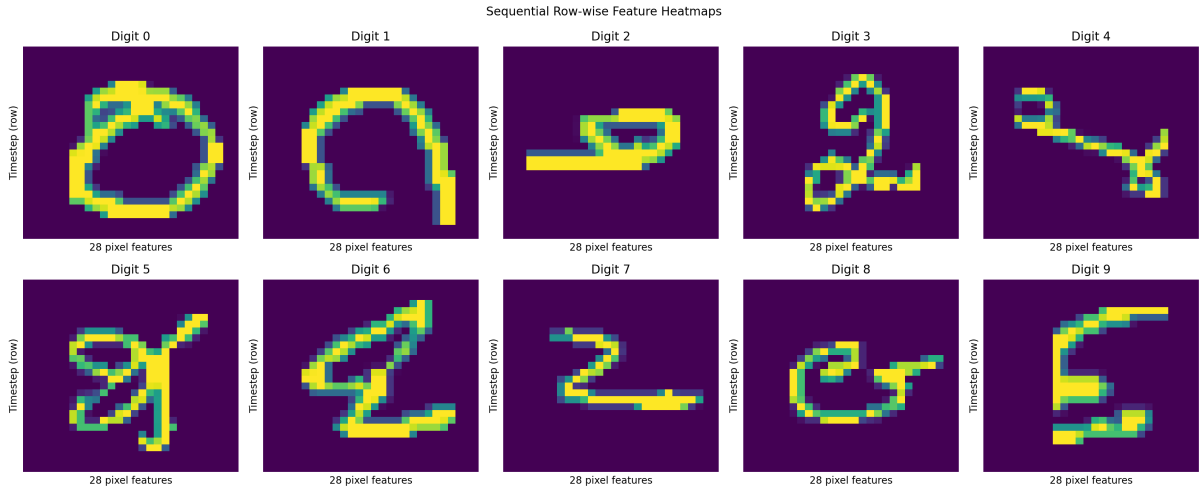


Figure 6: Row-wise sequential feature heatmaps for one test example from each digit class.

8.2 Comparative observations

Vanilla RNN. The two RNN configurations achieve test accuracies of 85.48% and 89.00%. The 128-unit model with dropout improves on the 64-unit model in both validation and test accuracy. This suggests that the wider configuration provides a more useful representation under the current training setup.

LSTM. The 64-unit LSTM achieves 90.30% test accuracy, the highest held-out test accuracy among the six configurations. Increasing the recurrent layer to 128 units and adding dropout improves validation accuracy to 90.76%, but its test accuracy is slightly lower at 89.91%.

GRU. The 64-unit GRU attains the highest validation accuracy (91.15%) and the highest training accuracy (94.36%). The 128-unit GRU with dropout performs less well on validation and test data in this experiment. Larger hidden size and dropout therefore do not consistently improve results; their impact depends on the recurrent architecture and learning rate.

Generalization. The selected GRU has a 4.78 percentage-point difference between its training accuracy (94.36%) and test accuracy (89.58%). This indicates a generalization gap, but the available metrics and curves alone do not establish severe overfitting. The strongest model depends on the evaluation criterion: validation accuracy selects the GRU, whereas test accuracy is highest for the 64-unit LSTM.

8.3 Gradient-flow considerations

The recurrent architectures use different mechanisms to carry information across time. A Vanilla RNN repeatedly applies recurrent transformations and can experience vanishing gradients when

learning dependencies over many time steps. LSTM introduces a cell state and input, forget, and output gates to regulate information and provide a more controlled route for gradient propagation. GRU uses update and reset gates with a simpler gated state update. These mechanisms make LSTM and GRU useful alternatives for sequential learning.

This experiment did not save gradient norms or other direct gradient-flow diagnostics. Consequently, this comparison is qualitative and based on architectural mechanisms; it does not claim to have measured a gradient-flow advantage for any particular model.

9 Discussion

Across the six configurations, validation accuracies range from 84.90% to 91.15%, while test accuracies range from 85.48% to 90.30%. The 64-unit GRU is selected because it has the highest validation accuracy. The 64-unit LSTM scores slightly higher on the held-out test set, illustrating why model selection should use validation data and preserve the test split for final assessment.

The error curves show rapid early improvement followed by slower changes. Increasing the number of recurrent units and adding dropout do not consistently improve accuracy. The 128-unit RNN improves over the 64-unit RNN, while the 128-unit GRU scores lower than the 64-unit GRU. Similarly, the 128-unit LSTM has higher validation accuracy but slightly lower test accuracy than its 64-unit counterpart.

The sequential heatmaps confirm that each input is processed as a sequence of image rows. Class-wise metrics show that model errors are not evenly distributed: class 6 has the weakest recall and is often classified as class 7. Future experiments could investigate longer training schedules, learning-rate schedules, multiple random seeds, alternative row/column scanning directions, bidirectional recurrent layers, or convolutional-recurrent models. These alternatives were not evaluated in the reported results and are proposed only as future work.

10 Conclusion

A row-wise recurrent pipeline was evaluated for ten-class handwritten Kannada digit classification. Six Vanilla RNN, LSTM, and GRU configurations were trained on normalized 28×28 inputs using a batch size of 128 and early stopping. The validation-selected model was `GRU_64_lr001`, which achieved 94.36% training accuracy, 91.15% validation accuracy, and 89.58% test accuracy. The `LSTM_64_lr001` configuration achieved the highest test accuracy, at 90.30%, among the six candidates.

The report includes the error curves for the tested configurations, sample digit visualizations, accuracy comparison tables, class-wise metrics, the complete test confusion matrix, and sequential feature heatmaps. Overall, the results demonstrate that recurrent architectures can model row-wise image sequences, while the comparison also shows that the best configuration depends on the model-selection criterion and that performance varies by digit class.

11 Key Experimental Values

Table 6: Summary of key experimental values.

Item	**Value**
Task	Sequential handwritten Kannada digit classification
Classes	10 (digits 0–9)
Labelled input size	10,240 samples
Image / sequence shape	28×28; 28 time steps and 28 features
Train / validation / test	7,168 / 1,536 / 1,536 (70% / 15% / 15%)
Normalization	Pixel intensity divided by 255.0
Batch size / maximum epochs	128 / 20
Early stopping	Validation loss; patience 4; restore best weights
Selected configuration	GRU; 64 units; dropout 0.0; learning rate 0.001
Selected model accuracy	Training 94.36%; validation 91.15%; test 89.58%
Highest test accuracy	LSTM; 64 units; dropout 0.0; learning rate 0.001; test 90.30%